

226. 翻转二叉树

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 广度优先搜索, 二叉树
- **Companies:** (Not Available)

给你一棵二叉树的根节点 `root`，翻转这棵二叉树，并返回其根节点。

示例 1:

输入：root = [4,2,7,1,3,6,9]

输出：[4,7,2,9,6,3,1]

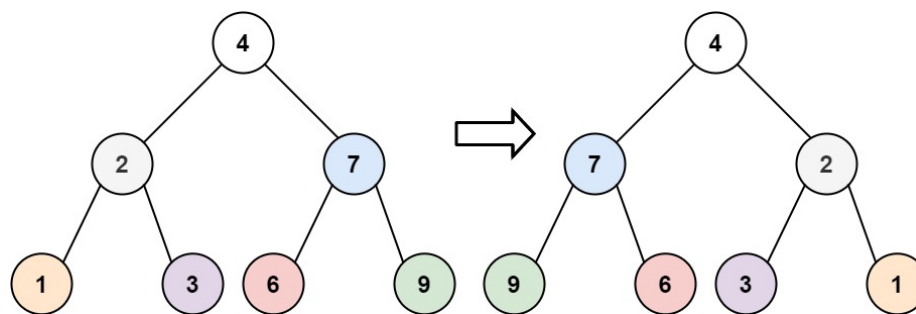


Figure 1: img

示例 2:

输入：root = [2,1,3]

输出：[2,3,1]

示例 3:

输入：root = []

输出：[]

提示:

- 树中节点数目范围在 $[0, 100]$ 内
- $-100 \leq \text{Node.val} \leq 100$

Discussion | Solution

解决方法

1. 问题理解

目标：翻转（或称为镜像、反转）给定的二叉树。**操作：**对于树中的每一个节点，交换其左子节点和右子节点。

2. 思路分析

方法一：递归（深度优先搜索 - DFS） 翻转操作具有递归性质：要翻转整棵树，只需先递归地翻转根节点的左右子树，然后交换根节点的左右子节点指针即可。

1. **基本情况 (Base Case)：**如果当前节点 `root` 为 `null`，表示到达空节点，无需翻转，直接返回 `null`。
2. **递归步骤 (Recursive Step)：**
 - 递归地翻转左子树：`invertTree(root.left)`。递归调用会处理好左子树内部的翻转。
 - 递归地翻转右子树：`invertTree(root.right)`。递归调用会处理好右子树内部的翻转。
 - **交换当前节点 `root` 的左右子节点指针：**
`temp = root.left`
`root.left = root.right`
`root.right = temp`
或者直接 `root.left, root.right = root.right, root.left`。
 - 返回当前节点 `root`（其左右子树已被交换）。

注意：交换操作可以在递归调用之前或之后进行，效果相同。通常放在递归调用之后（后序遍历位置）更符合“先处理子问题再合并”的思路，但放在前面（前序遍历位置）也可以。

执行流程示例 ([4,2,7,1,3,6,9]) - 后序交换： 1. `invertTree(4)` 2. `-> invertTree(2)` 3. `-> invertTree(1)` `-> 返回 null (左), 返回 null (右)`, 交换 `null` 和 `null`, 返回 1 4. `-> invertTree(3)` `-> 返回 null (左), 返回 null (右)`, 交换 `null` 和 `null`, 返回 3 5. `-> 节点 2 收到翻转后的 1 (还是 1) 和 3 (还是 3)`。交换 `root(2).left (1)` 和 `root(2).right (3)`。节点 2 变为 `[2,3,1]`。返回 2。6. `-> invertTree(7)` 7. `-> invertTree(6)` `-> 返回 6` 8. `-> invertTree(9)` `-> 返回 9` 9. `-> 节点 7 收到翻转后的 6 和 9`。交换 `root(7).left (6)` 和 `root(7).right (9)`。节点 7 变为 `[7,9,6]`。返回 7。10. 节点 4 收到翻转后的 2 (`[2,3,1]`) 和 7 (`[7,9,6]`)。交换 `root(4).left (2)` 和 `root(4).right (7)`。节点 4 变为 `[4,7,2]` 且子树已翻转。返回 4。

方法二：迭代（广度优先搜索 - BFS / 层序遍历） 我们也可以使用迭代的方式，逐层或逐个节点地进行翻转。BFS 是一个不错的选择。

1. **初始化：**
 - 如果 `root` 为 `null`，返回 `null`。
 - 创建一个队列 `queue`，并将根节点 `root` 入队。
2. **遍历队列：**
 - 当队列不为空时，进行循环：
 - 将队首节点 `node` 出队。

- 交换当前节点 `node` 的左右子节点：
`temp = node.left`
`node.left = node.right`
`node.right = temp`
 - 如果交换后的左子节点（原右子节点）不为空，将其入队。
 - 如果交换后的右子节点（原左子节点）不为空，将其入队。
3. 返回结果：循环结束后，整棵树已被翻转，返回原始根节点 `root`。

执行流程示例 ([4,2,7,1,3,6,9]):

步骤	queue	出队 node	操作	树状态 (部分)
1	[4]	-	初始化	[4,2,7]
2	[]	4	交换 2 和 7. <code>node(4).left=7,</code> <code>node(4).right=2</code>	[4,7,2]
3	[7, 2]	4	入队 7, 2	[4,7,2]
4	[2]	7	交换 6 和 9. <code>node(7).left=9,</code> <code>node(7).right=6</code>	[4,7(9,6),2]
5	[2, 9, 6]	7	入队 9, 6	[4,7(9,6),2]
6	[9, 6]	2	交换 1 和 3. <code>node(2).left=3,</code> <code>node(2).right=1</code>	[4,7(9,6),2(3,1)]
7	[9, 6, 3, 1]	2	入队 3, 1	[4,7(9,6),2(3,1)]
8	[6, 3, 1]	9	交换 null 和 null	...
9	[6, 3, 1]	9	无子节点入队	...
10	[3, 1]	6	交换 null 和 null	...
11	[3, 1]	6	无子节点入队	...
12	[1]	3	交换 null 和 null	...
13	[1]	3	无子节点入队	...
14	[]	1	交换 null 和 null	...
15	[]	1	无子节点入队	...
16	[]	-	队列为空，循环结束，返回 root (4)	[4,7(9,6),2(3,1)]

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import Optional
import collections
```

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一: 递归 (DFS)
    def invertTreeRecursive(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        """
        使用递归 (DFS) 翻转二叉树
        """
        # 基本情况: 空节点无需翻转
        if not root:
            return None

        # 递归翻转左子树
        self.invertTreeRecursive(root.left)
        # 递归翻转右子树
        self.invertTreeRecursive(root.right)

        # 交换当前节点的左右子节点
        root.left, root.right = root.right, root.left

        # 返回翻转后的当前节点
        return root

    # 方法二: 迭代 (BFS)
    def invertTreeIterative(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        """
        使用迭代 (BFS) 翻转二叉树
        """
        if not root:
            return None

        queue = collections.deque([root])
        while queue:
            node = queue.popleft()

            # 交换左右子节点
            node.left, node.right = node.right, node.left

            # 如果子节点存在, 则将其加入队列继续处理
            if node.left:
                queue.append(node.left)

```

```

        if node.right:
            queue.append(node.right)

    return root # 返回原始根节点, 其结构已被修改

# 最终提交可以选择其中一种方法
def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
    # return self.invertTreeRecursive(root)
    return self.invertTreeIterative(root)

```

4. 复杂度分析

- **递归方法 (DFS):**
 - 时间复杂度: $O(N)$, 其中 N 是树中的节点数。每个节点被访问和操作 (交换) 一次。
 - 空间复杂度: $O(H)$, 其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下 (链状树) 为 $O(N)$, 平均情况下 (平衡树) 为 $O(\log N)$ 。
- **迭代方法 (BFS):**
 - 时间复杂度: $O(N)$ 。每个节点入队出队各一次, 并进行一次交换操作。
 - 空间复杂度: $O(W)$, 其中 W 是树的最大宽度。在最坏情况下 (完全二叉树的最后一层) 可能达到 $O(N)$ 。

5. 如何在面试中讲解

1. **明确目标:**
 - “目标是翻转二叉树, 即对每个节点, 交换其左右子节点。”
2. **递归思路 (DFS):**
 - “这是一个典型的可以用递归解决的问题。要翻转一棵树, 我们可以先递归地翻转它的左右子树, 然后交换根节点的左右孩子指针。”
 - “讲解递归三部曲: **终止条件** (节点为空, 返回 null), **递归左右子树**, **处理当前节点** (交换左右指针)。”
 - “可以讨论交换操作放在递归调用之前或之后都可以。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(H)$ 。”
3. **迭代思路 (BFS):**
 - “也可以用迭代的方式, 比如层序遍历。遍历树中的每个节点, 并交换其左右子节点。”
 - “使用队列实现 BFS。每次从队列中取出一个节点, 交换其左右孩子, 然后将非空的孩子 (交换后的) 加入队列。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(W)$ 。”
4. **对比与选择:**
 - “递归代码通常更简洁。BFS 迭代可以避免递归深度问题。”
 - “两种方法的时间复杂度都是 $O(N)$, 空间复杂度取决于树的形状。”
5. **边界条件:**
 - “需要处理空树 (root 为 null) 的情况, 直接返回 null。”
 - “叶子节点的左右子节点都是 null, 交换后仍然是 null, 逻辑正确。”